QUESTION 1:

Aim :

To analyze the efficiency of the Merge Sort algorithm under best, average, and worst-case scenarios using asymptotic notations.

```
def merge(left, right):
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result
def merge_sort(arr):
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])
    return merge(left, right)
arr = list(map(int, input("Enter numbers separated by space: ").split()))
print("Original Array:", arr)
sorted_array = merge_sort(arr)
print("Sorted Array:", sorted_array)
```

```
Enter numbers separated by space: 45 12 67 3 89 23
Original Array: [45, 12, 67, 3, 89, 23]
Sorted Array: [3, 12, 23, 45, 67, 89]
```

Analysis:

- **Best Case:**

  Input Condition: The array is already sorted.

  Example:1 2 3 4 5 6

  Behavior: Merge Sort still divides the array into two halves and merges them.

  Complexity : O(n log n)

- **Average Case**

  Input Condition : Randomly ordered data.

  Example : 8 2 5 1 7 3

  Complexity : O(n log n)

- **Worst Case**

  Input Condition : Elements arranged so maximum

  comparisons occur during merging.

  Example: 6 5 4 3 2 1

  Complexity : O(n log n)

Complexity Table:

Case	Input Condition	Time Complexity
Best	Already Sorted	O(n log n)
Average	Random Order	O(n log n)
Worst	Reverse Order	O(n log n)

Space Complexity : O(n)

Because Merge Sort requires additional memory for merging.

Asymptotic Notation:

Notation	Meaning	Merge Sort
Big-O	Upper Bound	O(n log n)
Big-Theta	Exact Bound	Θ(n log n)
Big-Omega	Lower Bound	Ω(n log n)

Comparison Chart:

Case	Complexity
Best	O(n log n)
Average	O(n log n)
Worst	O(n log n)

Practical Relevance:

The average case is the most relevant because real-world data is usually unsorted or randomly ordered. Merge Sort consistently performs in O(n log n) regardless of the input arrangement, making it suitable for applications such as databases, search engines, and large-scale data processing.

Conclusion:

Merge Sort provides consistent performance in all cases with Θ(n log n) time complexity. Although it requires additional memory, it is highly efficient and stable for sorting large datasets.